

Food Menu Connector Architecture (v1)

Purpose

This document defines the architecture for the **Food Menu Connector**, the first end-to-end connector in the AI Campus Copilot system.

This connector is intentionally built first because it is:

- simple
- easy to test
- representative of the overall connector architecture
- useful for validating our asynchronous sync design

The Food Connector will act as the **template architecture** for future connectors:

- ERP
- LMS
- Timetable
- Notices
- Attendance
- Events

The goal is:

Build one connector correctly → scale architecture to all future connectors.

Product Context

Students use multiple disconnected systems inside a campus.

Example:

- ERP
- LMS
- Mess portal
- Timetable portal
- Club announcements

Our system aims to provide:

One AI-powered campus assistant

The assistant should answer questions naturally:

Examples:

"What's for lunch?"

"What's dinner today?"

"Should I eat before class?"

Instead of calling campus systems live every time, we use an:

asynchronous cached architecture

This improves:

- speed
- reliability
- extensibility
- proactive intelligence

Design Philosophy

The architecture follows these principles:

1. Connector-based design

Each feature is a **connector module**.

Example:

Food Connector
ERP Connector
LMS Connector

Every connector works independently.

This allows:

- modular development
- independent team work
- easy scaling

- low merge conflicts
-

2. Connector ownership

Each connector owns:

- its fetching logic
- parsing logic
- sync logic
- database tables

Example:

Food Connector owns:

```
meals
```

ERP connector later will own:

```
attendance  
schedule
```

This prevents engineering conflicts.

3. Normalization-first architecture

Different campuses may expose food menus differently.

Example:

Campus A JSON

```
{  
  "breakfast": ["idli"],  
  "lunch": ["rice", "paneer"]  
}
```

Campus B API

```
{
  "menu": {
    "morning": {
      "items": ["poha"]
    }
  }
}
```

Campus C website

HTML page with visible menu text.

Instead of storing these formats directly, the system converts everything into:

one internal normalized schema

Example normalized output:

```
{
  "meal_type": "lunch",
  "items": [
    "rice",
    "paneer"
  ]
}
```

This ensures:

AI never needs to understand campus-specific formats.

Connector Modes

The Food Connector supports 3 ingestion modes.

Mode 1 — Default JSON

Used when campus has no menu system.

We provide:

```
default_menu.json
```

Example:

```
{  
  "breakfast": ["idli", "tea"],  
  "lunch": ["rice", "paneer"],  
  "dinner": ["dal", "roti"]  
}
```

Mode 2 — Campus API

Used when campus already exposes menu data via API.

Example:

```
https://campus.edu/api/menu
```

The connector fetches JSON.

Then normalizes it.

Mode 3 — Website Scraper

Used when menu exists only as a webpage.

Example:

```
https://campus.edu/mess-menu
```

The connector scrapes the page.

Extracts food data.

Then normalizes it.

High-Level Architecture

Campus Menu Source
(Default JSON / API / Website)

↓

Food Connector

Fetcher Layer

↓

Parser Layer

↓

Normalizer Layer

↓

Supabase Database

↓

AI Query Layer

↓

Student Query

End-to-End Flow

Step 1 — Student configures connector

Student selects:

Food Menu Connector

Chooses:

default
OR

```
campus website  
OR  
campus API
```

Configuration is stored in:

```
connector_configs
```

Step 2 — Sync Worker runs asynchronously

A background sync process runs periodically.

Example:

Every 15 minutes.

The worker:

1. reads connector config
2. fetches source data
3. parses data
4. normalizes format
5. stores menu locally

Step 3 — Data stored in database

Normalized menu enters:

```
meals
```

table.

Example:

```
{  
  "date": "2025-08-05",  
  "meal_type": "lunch",  
  "items": [  
    "paneer",  
    "rice",  
  ]  
}
```

```
    "roti"  
  ]  
}
```

Raw campus payload is also stored for debugging.

Step 4 — Student asks question

Student:

"What's for lunch?"

AI does NOT call campus website.

Instead:

AI queries local DB.

Example query:

```
SELECT items  
FROM meals  
WHERE date = TODAY  
AND meal_type = 'lunch'
```

AI responds:

Lunch today is paneer, rice, and roti.

Why Async Architecture?

We intentionally avoid live fetching.

Bad approach:

```
Student asks  
↓  
ERP call  
↓  
Website call
```


↓
Slow response

Problems:

- latency
- website failures
- API limits
- poor demo quality

Instead:

Good approach:

Background sync
↓
Local database
↓
Instant AI response

Benefits:

- fast
- reliable
- scalable
- enables proactive reminders later

Database Design

We use:

Supabase (PostgreSQL)

Reason:

- relational structure fits future campus systems
 - supports JSON storage
 - scalable
 - future RAG support using pgvector
 - hackathon-friendly
-

Tables

connector_configs

Stores:

how connector works

Attributes:

```
id
user_id
connector_name
connector_mode
source_url
refresh_interval
config_json
is_active
created_at
```

meals

Stores:

normalized food menu

Attributes:

```
id
user_id
date
meal_type
items
source
raw_payload
synced_at
```

Modularity

Every connector behaves like a mini-project.

Example structure:

```
connectors/  
  
  mess/  
    fetcher.py  
    parser.py  
    normalizer.py  
    sync.py  
    schema.py  
  
  erp/  
    ...  
  
  lms/  
    ...
```

Each connector owns:

- fetching
- parsing
- tables
- synchronization

Shared system components:

```
AI layer  
database connection  
scheduler  
connector registry
```

This enables parallel team development.

Example:

Student A:

works on:

```
mess connector
```

Friend:

works on:

erp connector

No merge conflicts.

Future Scalability

Once Food Connector works:

Future connectors follow same pattern:

```
Fetch
↓
Parse
↓
Normalize
↓
Store locally
↓
AI query
```

This becomes the standard connector architecture for the entire Campus Copilot system.